

Face Mask Detection

An Application of Deep Learning & Computer Vision

Based on the provided Python implementation

Introduction to Key Concepts



Artificial Intelligence

Creating systems that simulate human intelligence to perform tasks like problem-solving and decision-making.



Machine Learning (ML)

A subset of AI where systems "learn" directly from data and identify patterns without explicit programming.



Deep Learning

A subset of ML that uses multi-layered "neural networks" to solve complex problems, like image recognition.



Computer Vision

A field of AI that trains computers to interpret and understand the visual world, which is the core of this project.

Project Aim and Motivation

Aim

To build and deploy a deep learning model that can accurately detect and classify face mask usage ('Mask' or 'No Mask') in a real-time video stream.

Motivation

To gain practical experience in the end-to-end machine learning lifecycle: from data preprocessing and model training to real-time inference and deployment using a live webcam feed.

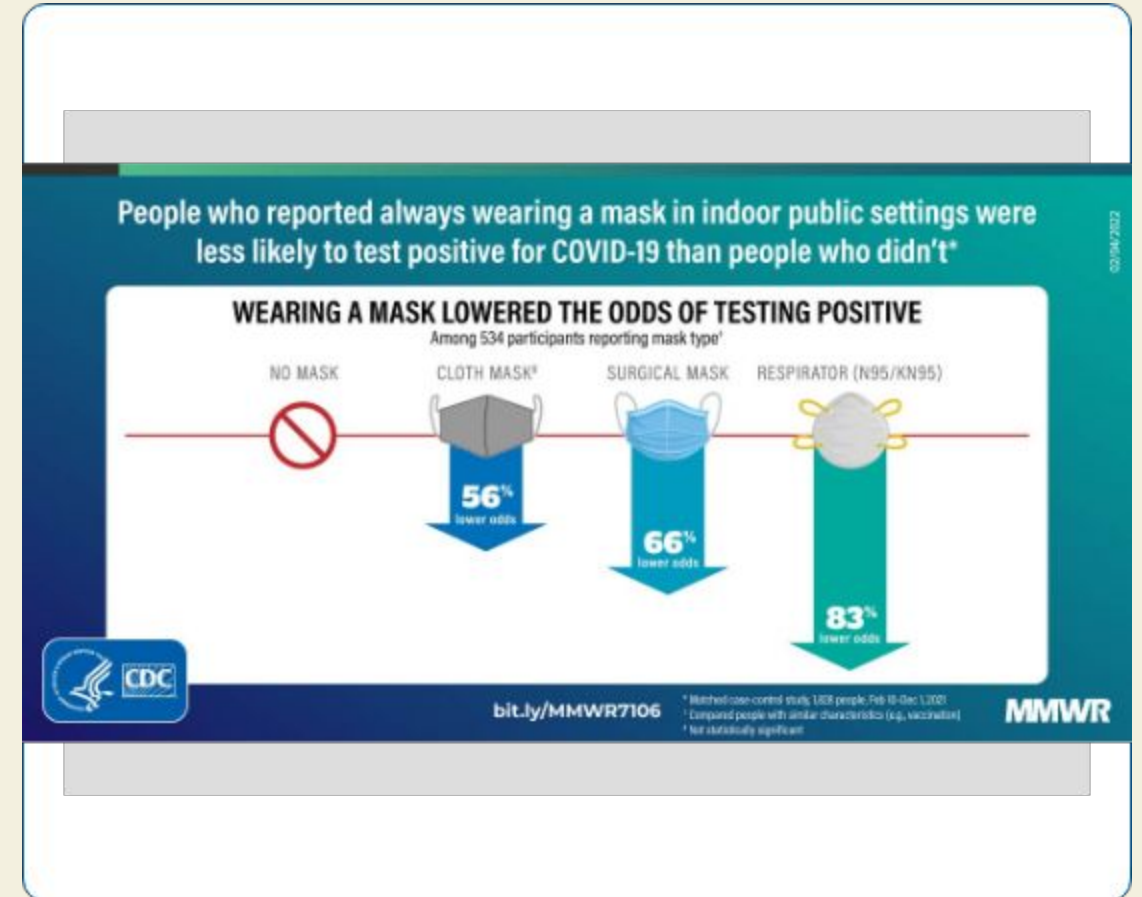
Problem Statement

The Challenge

This project solves a two-part problem for every single frame of video:

- ➔ **Face Localization:** Where are the faces in the image? The system must be robust to different head poses, lighting, and occlusions.
- ➔ **Mask Classification:** For each detected face, is the person wearing a mask or not?

This entire process must be computationally efficient to run in real-time (many frames per second).



Key Technologies Used

Core Components

Our implementation relies on several powerful, pre-existing tools:

- ➔ **OpenCV:** Used for all real-time video capture, image processing (resizing, color conversion), and drawing the final bounding boxes.
- ➔ **Caffe Face Detector:** A pre-trained SSD (Single Shot Detector) model used as our `faceNet` to perform fast and efficient *face localization*.
- ➔ **TensorFlow & Keras:** The core deep learning libraries used to build, train, and load our custom `maskNet` classifier.

Methodology (Project Workflow)

1. Data Prep

Load and augment image data using ``ImageDataGenerator`` (rotation, zoom, flip) from the ``train.py`` script.

2. Model Training

Train a new classifier head on top of frozen ``MobileNetV2`` base. Save as ``mask_detector.model.h5``.

3. Face Detection

In ``detect.py``, use the pre-trained ``faceNet`` (Caffe SSD) to find all faces in the live video frame.

4. Mask Prediction

Pass each detected face (ROI) to the trained ``maskNet``, which classifies "Mask" or "No Mask" in real-time.

Data and Preprocessing

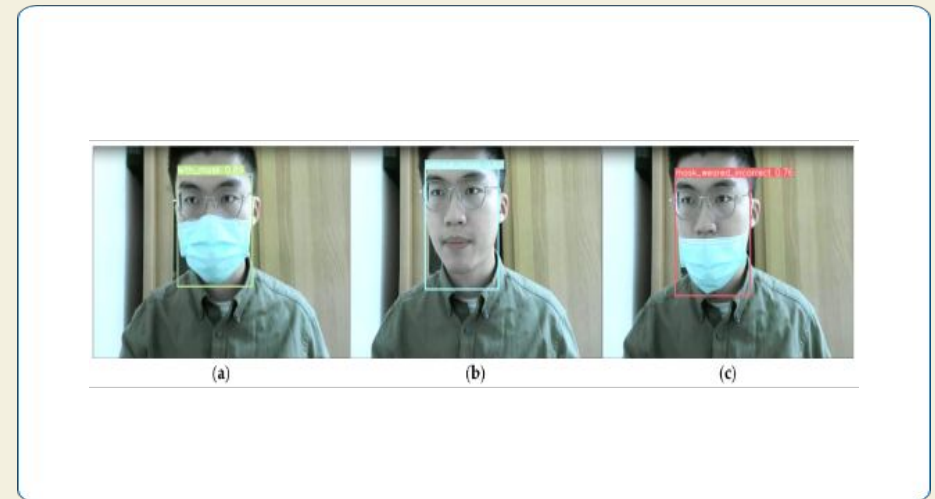
Dataset

The model was trained on a dataset of images with two categories: "with_mask" and "without_mask".

Data Augmentation

To prevent overfitting and improve generalization, we used `ImageDataGenerator` with:

- ➔ Rotation, Zoom, and Shear
- ➔ Horizontal and Vertical Flips
- ➔ Automatic 80/20 Train/Validation Split
- ➔ `MobileNetV2.preprocess\_input` for correct normalization.



Model Architecture (Transfer Learning)

1. Base Model (Frozen)

We loaded `MobileNetV2` pre-trained on ImageNet. Its layers are "frozen" (`layer.trainable = False`) to use its learned features.

2. Custom Head (Trainable)

We added a new "head" to the model:

- ➔ `AveragePooling2D` to condense features.
- ➔ `Flatten` to prepare for dense layers.
- ➔ `Dense (128, 'relu')` as a hidden layer.
- ➔ `Dropout (0.5)` to prevent overfitting.
- ➔ `Dense (2, 'softmax')` for the final "Mask" / "No Mask" classification.

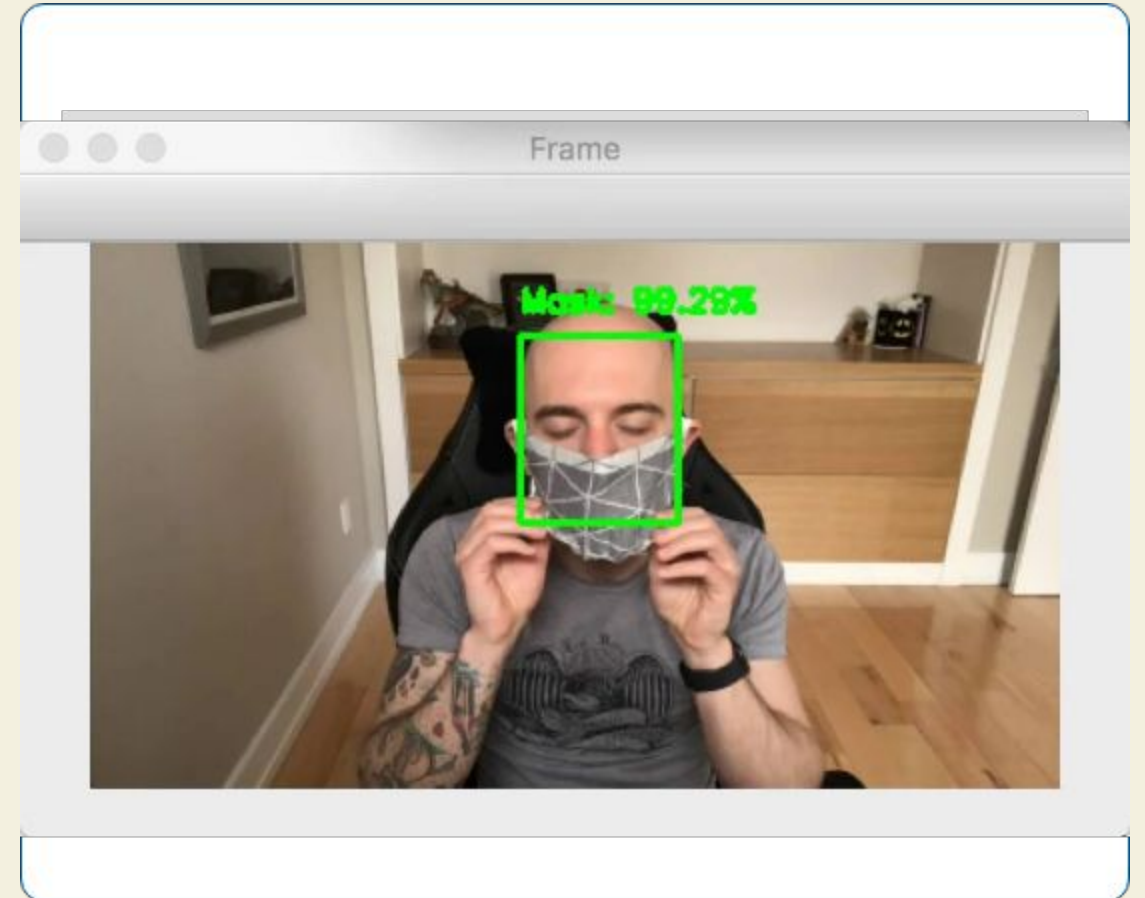
Training Results

Performance Metrics

The model was trained with the following parameters from `train.py`:

- ➔ **Learning Rate:** `1e-4` (0.0001)
- ➔ **Epochs:** 20
- ➔ **Batch Size:** 32
- ➔ **Optimizer:** Adam

The resulting plot shows the model trained perfectly. The validation accuracy (gray line) starts high and stays high, ending around **98-99%**, matching the training accuracy. This indicates a well-generalized model with no overfitting.



Detection Output and Conclusion

Real-Time Output

The final script (`detect.py`) successfully loads both models and draws colored bounding boxes:

➔ **Green Box:** `mask > withoutMask`

➔ **Red Box:** `mask < withoutMask`

Conclusion

This project was a complete success. We built a highly accurate, real-time mask detector by combining a pre-trained face detector with a custom-trained classifier built using transfer learning.